

TOAE209/TOAH209 – Design and Analysis of Algorithms

Week 13: PSPACE and Extending Tractability

Foreign Trade University

Contents

1	Introduction: Beyond NP	1
2	PSPACE and Quantified Boolean Formulas	1
2.1	The class PSPACE	1
2.2	Quantified Boolean Formulas (QBF / QSAT)	2
3	Games as PSPACE Problems	2
3.1	Game trees and minimax	2
3.2	Alpha-beta pruning	3
4	Extending Tractability I: Fixed-Parameter Tractability	3
4.1	Bounded search trees: Vertex Cover in $O(2^k n)$	4
4.2	Kernelization	5
5	Extending Tractability II: Exploiting Special Structure	6
5.1	Independent Set on trees via dynamic programming	6
6	Extending Tractability III: Branch and Bound	6
6.1	Branch-and-bound for the 0/1 Knapsack	7
6.2	Branch-and-bound for the TSP	7
7	A Polynomial Island: 2-SAT	7
8	State-Space Search: Iterative Deepening	8
9	Set Cover by FPT/Bitmask Search	9
10	Summary and Looking Ahead	9

1 Introduction: Beyond NP

The previous weeks built up a sharp picture of the world of computation: some problems are solvable in polynomial time (the class **P**), many natural problems are **NP-complete** and (conjecturally) admit no polynomial-time algorithm, and reductions let us relate one hard problem to another. This week we push the picture in two directions.

- **Upward**, to a larger complexity class: **PSPACE**, the problems solvable using a polynomial amount of *memory* (regardless of time). PSPACE contains all of NP, and its canonical complete problem – deciding a *quantified* boolean formula (QSAT/QBF) – captures the flavor of *games*, where two players alternate moves. Game playing is naturally PSPACE-shaped: “does there *exist* a move for me such that for *all* replies of my opponent, there exists a move for me, . . . , I win?”
- **Sideways**, by *extending tractability*: even when a problem is NP-hard in general, we can often solve the instances we actually care about, *provably* and *efficiently*, by exploiting (a) a small *parameter*, (b) special *structure* in the input, or (c) clever *pruning* of an exponential search. These are the themes of fixed-parameter tractability (FPT), dynamic programming on special graphs, and branch-and-bound.

This follows Kleinberg & Tardos, Chapters 9 (PSPACE) and 10 (extending the limits of tractability). The unifying message: *NP-hardness is a worst-case statement*. It does not forbid us from solving real instances; it just tells us where to look for the leverage that makes a particular family of instances tractable.

2 PSPACE and Quantified Boolean Formulas

2.1 The class PSPACE

Definition 2.1 (PSPACE). *A decision problem is in **PSPACE** if there is an algorithm that decides it using an amount of working memory (space) bounded by a polynomial in the input size n . There is no restriction on running time (it may be exponential), only on space.*

Two basic facts position PSPACE relative to the classes we already know.

Theorem 2.2. $P \subseteq NP \subseteq PSPACE$.

Proof sketch. $P \subseteq NP$ is immediate. For $NP \subseteq PSPACE$: an NP problem can be decided by trying every candidate certificate in turn and running the (polynomial-time) verifier on each. We can enumerate certificates one at a time, reusing the same polynomial-size memory for each verification, discarding it before the next – so total *space* stays polynomial even though we examine exponentially many certificates over exponential *time*. $\square$

The reuse-of-space idea is the heart of PSPACE: *time can blow up exponentially, but space is recycled*. It is widely believed (but unproven) that $NP \neq PSPACE$, so PSPACE is plausibly strictly larger than NP.

2.2 Quantified Boolean Formulas (QBF / QSAT)

Recall that SAT asks whether *there exists* a satisfying assignment: $\exists x_1 \exists x_2 \cdots \exists x_n \Phi(x_1, \dots, x_n)$. QSAT generalizes this by allowing *alternating* quantifiers.

Definition 2.3 (QSAT / TQBF). *A quantified boolean formula has the form*

$$Q_1 x_1 Q_2 x_2 \cdots Q_n x_n \Phi(x_1, \dots, x_n),$$

*where each $Q_i \in \{\exists, \forall\}$ and Φ is a boolean formula (often in CNF). The **QSAT** (also **TQBF**, “true QBF”) problem is to decide whether the fully-quantified formula evaluates to **true**.*

A fully-quantified formula has a definite truth value (no free variables). It is evaluated by the obvious recursion: $\exists x_i \Psi$ is true iff Ψ is true for $x_i = \mathbf{false}$ or for $x_i = \mathbf{true}$; $\forall x_i \Psi$ is true iff Ψ is true for *both* values. Problem 1 asks you to implement exactly this recursive evaluator.

Theorem 2.4 (Stockmeyer–Meyer). *QSAT is PSPACE-complete.*

Why it is in PSPACE. The recursive evaluator above uses a recursion stack of depth n (one frame per quantifier), each frame storing $O(1)$ extra information (the current value being tried for one variable). At a leaf we evaluate Φ on a complete assignment in polynomial space. Although the recursion explores a tree of up to 2^n leaves (exponential *time*), at any instant only one root-to-leaf path is materialized, so the *space* is $O(n + |\Phi|)$ – polynomial. Completeness (that every PSPACE problem reduces to QSAT) is shown by encoding a polynomial-space computation as a quantified formula; we omit the construction (Kleinberg & Tardos, Section 9.4). $\square$

The alternation $\exists\forall\exists\forall\cdots$ is exactly the logical structure of a two-player game, which we turn to next.

3 Games as PSPACE Problems

3.1 Game trees and minimax

Consider a finite, two-player, zero-sum, perfect-information game (tic-tac-toe, Nim, chess on a bounded board, ...). Players alternate moves; one player (**Max**) wants to maximize the final payoff and the other (**Min**) wants to minimize it. The set of all play sequences forms a **game tree**: internal nodes are positions (alternating between “Max to move” and “Min to move”), edges are moves, and leaves carry payoffs.

Definition 3.1 (Minimax value). *The minimax value of a position is defined recursively: a leaf’s value is its payoff; a Max node’s value is the maximum of its children’s values; a Min node’s value is the minimum of its children’s values. The minimax value of the root is the payoff achievable under optimal play by both sides.*

Algorithm 1 Minimax (value for the player to move)

```

1: function MINIMAX(node, maximizing)
2:   if node is a leaf then return payoff(node)
3:   end if
4:   if maximizing then
5:     return  $\max_{c \in \text{children}(\text{node})}$  MINIMAX(c, false)
6:   else
7:     return  $\min_{c \in \text{children}(\text{node})}$  MINIMAX(c, true)
8:   end if
9: end function

```

The recursion mirrors a QBF exactly: “does Max have a move ($\exists$) so that for all Min replies ($\forall$) Max still has a move ($\exists$), ..., ending in a win?” This is why deciding the winner of a generalized board game is typically PSPACE-complete. Problem 2 asks you to implement minimax on a nested-list game tree.

3.2 Alpha-beta pruning

Plain minimax explores the entire tree. **Alpha-beta pruning** computes the *same* minimax value while skipping subtrees that cannot affect the result. It carries two bounds down the recursion:

- α = the best (largest) value Max can already guarantee along the current path;
- β = the best (smallest) value Min can already guarantee.

Whenever $\alpha \geq \beta$, the current node's remaining children are irrelevant – the opponent already has a better option elsewhere – so we *prune*.

Algorithm 2 Alpha-Beta Pruning

```

1: function ALPHABETA(node, maximizing,  $\alpha$ ,  $\beta$ )
2:   if node is a leaf then return payoff(node)
3:   end if
4:   if maximizing then
5:      $v \leftarrow -\infty$ 
6:     for each child c of node do
7:        $v \leftarrow \max(v, \text{ALPHABETA}(c, \text{false}, \alpha, \beta))$ 
8:        $\alpha \leftarrow \max(\alpha, v)$ 
9:       if  $\alpha \geq \beta$  then break ▷  $\beta$  cut-off
10:      end if
11:    end for
12:    return  $v$ 
13:   else
14:      $v \leftarrow +\infty$ 
15:     for each child c of node do
16:        $v \leftarrow \min(v, \text{ALPHABETA}(c, \text{true}, \alpha, \beta))$ 
17:        $\beta \leftarrow \min(\beta, v)$ 
18:       if  $\beta \leq \alpha$  then break ▷  $\alpha$  cut-off
19:      end if
20:    end for
21:    return  $v$ 
22:   end if
23: end function

```

Theorem 3.2. *Alpha-beta pruning returns the same value as plain minimax.*

Proof idea. A subtree is pruned only when its parent already has a sibling value forcing $\alpha \geq \beta$. In that situation, the value of the pruned subtree could only move the node's max/min in a direction the ancestor on the other side would never choose, so it cannot change any value on the path to the root. Hence the root value is unaffected by the pruning. $\square$

With an ideal move ordering, alpha-beta examines only $O(b^{d/2})$ leaves instead of $O(b^d)$ – effectively doubling the search depth reachable in the same time. Problem 3 asks you to implement alpha-beta and verify both that it returns the same value as minimax and that it visits strictly fewer leaves.

4 Extending Tractability I: Fixed-Parameter Tractability

NP-hardness is a worst-case statement over *all* inputs of size n . Often the inputs we care about have a small *parameter* k – the size of the solution, the treewidth, the number of clusters – even when n is large. FPT isolates the exponential blow-up in k alone.

Definition 4.1 (Fixed-parameter tractable). *A parameterized problem (with input size n and parameter k) is **fixed-parameter tractable (FPT)** if it can be solved in time $f(k) \cdot \text{poly}(n)$ for some function f depending only on k . The exponential cost (if any) is confined to $f(k)$; the dependence on n stays polynomial.*

The point: if k is small (say ≤ 20) then $f(k)$ – even 2^k – is a fixed constant, and the algorithm is genuinely efficient on large n , in stark contrast to a 2^n brute force.

4.1 Bounded search trees: Vertex Cover in $O(2^k n)$

Definition 4.2 (Vertex Cover). *A vertex cover of a graph $G = (V, E)$ is a set $C \subseteq V$ such that every edge has at least one endpoint in C . The decision problem: given G and an integer k , is there a vertex cover with $|C| \leq k$?*

Vertex Cover is NP-complete, but parameterized by k it is FPT, via a beautiful **bounded search tree**. The key observation:

Lemma 4.3 (Branching rule). *For any edge (u, v) , every vertex cover must contain u or v (or both).*

So we pick any uncovered edge (u, v) and *branch*: either $u \in C$ or $v \in C$. Each branch fixes one vertex and decrements the budget k by one, giving a search tree of depth $\leq k$ and branching factor 2.

Algorithm 3 FPT Vertex Cover (bounded search tree)

```

1: function VC( $E, k, C$ )
2:   if  $E = \emptyset$  then return  $C$  ▷ all edges covered
3:   end if
4:   if  $k = 0$  then return NONE ▷ budget exhausted, edges remain
5:   end if
6:   pick any edge  $(u, v) \in E$ 
7:   for  $w \in \{u, v\}$  do
8:      $E' \leftarrow$  edges of  $E$  not incident to  $w$ 
9:      $R \leftarrow \text{VC}(E', k - 1, C \cup \{w\})$ 
10:    if  $R \neq \text{NONE}$  then return  $R$ 
11:    end if
12:  end for
13:  return NONE
14: end function

```

Theorem 4.4. *The bounded-search-tree algorithm decides whether a vertex cover of size $\leq k$ exists in time $O(2^k \cdot (n + m))$, and returns such a cover if one exists.*

Proof. Correctness. The recursion explores all ways of choosing, for each edge encountered, which endpoint enters the cover. By the branching lemma, every vertex cover is consistent with at least one such sequence of choices, so if a cover of size $\leq k$ exists, some leaf of the search tree at depth $\leq k$ finds it; conversely every returned set covers all edges (we only return when $E = \emptyset$) and has size $\leq k$ (each recursive level adds one vertex and decrements k). **Running time.** The recursion tree has depth $\leq k$ and branching factor 2, hence $\leq 2^k$ leaves and $O(2^k)$ nodes; each node does $O(n + m)$ work to filter edges. Total $O(2^k(n + m))$. $\square$

Note the contrast with the $\binom{n}{k}$ brute force over all k -subsets: the search tree depends *exponentially only on k* . Problems 4–5 ask you to implement this decider and verify it against a brute-force minimum vertex cover on small graphs.

4.2 Kernelization

A second pillar of FPT is **kernelization**: in polynomial time, shrink the instance to an equivalent “kernel” whose size depends only on k , then run any (even brute-force) algorithm on the small kernel.

For Vertex Cover, two reduction rules suffice for a first kernel:

- **High-degree rule.** If a vertex v has degree $> k$, then v must be in every cover of size $\leq k$. (If v were *not* in C , then all $> k$ of its incident edges would need their *other* endpoints in C , forcing $|C| > k$.) So put v in the cover, delete v and its edges, and decrement k .
- **Isolated-vertex rule.** A degree-0 vertex covers no edge and is never needed; remove it.

Theorem 4.5 (Kernel size). *After applying the two rules exhaustively, a yes-instance has at most k^2 edges and at most $k^2 + k$ non-isolated vertices.*

Proof. Once no high-degree vertex remains, every vertex has degree $\leq k$. A cover of size $\leq k$ touches every edge, and each of its $\leq k$ vertices covers $\leq k$ edges, so there are at most $k \cdot k = k^2$ edges. Each such edge has 2 endpoints, so at most $2k^2$ vertex-incidences and certainly $O(k^2)$ non-isolated vertices remain. $\square$

Thus the original n drops out entirely: brute force on the kernel costs only a function of k . Problem 6 asks you to implement these two reduction rules and return the reduced instance.

5 Extending Tractability II: Exploiting Special Structure

Even an NP-hard problem can become polynomial when restricted to inputs with special structure. The cleanest example is solving a hard graph problem on a **tree**.

5.1 Independent Set on trees via dynamic programming

Definition 5.1 (Independent Set). *An independent set of $G = (V, E)$ is a set $S \subseteq V$ with no edge between any two members. Maximum Independent Set asks for the largest such S . It is NP-hard on general graphs.*

On a *tree*, however, a root-to-leaf DP solves it in linear time. Root the tree anywhere and, for each vertex v , compute two quantities over the subtree rooted at v :

$$\text{incl}[v] = 1 + \sum_{c \in \text{children}(v)} \text{excl}[c], \quad \text{excl}[v] = \sum_{c \in \text{children}(v)} \max(\text{incl}[c], \text{excl}[c]).$$

The first is the best subtree solution *including* v (so no child may be included); the second is the best *excluding* v (each child is free to be included or not).

Algorithm 4 Maximum Independent Set on a Tree

```

1: root the tree at any vertex; compute a post-order of the vertices
2: for each vertex  $v$  in post-order (children before parents) do
3:    $\text{incl}[v] \leftarrow 1 + \sum_c \text{excl}[c]$ 
4:    $\text{excl}[v] \leftarrow \sum_c \max(\text{incl}[c], \text{excl}[c])$ 
5: end for
6: return  $\max(\text{incl}[\text{root}], \text{excl}[\text{root}])$ 

```

Theorem 5.2. *The DP computes a maximum independent set of an n -vertex tree in $O(n)$ time.*

Proof. **Correctness** is by induction on subtree size: $\text{incl}[v]$ and $\text{excl}[v]$ correctly record the best independent set of v 's subtree under the two cases for v , given (by induction) correct values for all children; including v forbids including any child (hence $\text{excl}[c]$), while excluding v leaves each child free (hence $\max$). The root case $\max(\text{incl}, \text{excl})$ allows or disallows the root. **Time:** each vertex is processed once, doing work proportional to its number of children; summing over the tree gives $O(n)$. $\square$

This is a template: many NP-hard problems become tractable on trees (and, more generally, on graphs of bounded *treewidth*) precisely because a tree admits such a clean subtree decomposition. Problem 7 asks you to implement the tree DP and verify it against a brute-force maximum independent set.

6 Extending Tractability III: Branch and Bound

Branch-and-bound keeps an exponential search structure but prunes aggressively using *bounds*: an optimistic estimate (upper bound for maximization, lower bound for minimization) of the best completion of a partial solution. If the bound cannot beat the best complete solution found so far (the *incumbent*), the whole subtree is discarded. The answer is always exact; only the search is faster.

6.1 Branch-and-bound for the 0/1 Knapsack

Definition 6.1 (0/1 Knapsack). *Given n items with values v_i and weights w_i , and capacity W , choose a subset maximizing total value subject to total weight $\leq W$.*

We branch on each item (include / exclude) and bound with the **fractional relaxation**: sort items by value-to-weight ratio, greedily take whole items, then a fraction of the next to fill the remaining capacity. This is the optimal value of the LP relaxation, hence a valid *upper bound* on any integral completion.

The optimum found must equal the value from the standard $O(nW)$ DP; Problem 8 asks you to verify this on random instances.

6.2 Branch-and-bound for the TSP

Definition 6.2 (Travelling Salesman Problem). *Given a symmetric distance matrix on n cities, find the shortest tour (cyclic permutation) visiting every city exactly once and returning to the start.*

Algorithm 5 Branch-and-Bound 0/1 Knapsack

```
1: sort items by  $v_i/w_i$  descending;  $best \leftarrow 0$ 
2: function EXPLORE( $i, curVal, curWt$ )
3:    $best \leftarrow \max(best, curVal)$ 
4:   if  $i = n$  then return
5:   end if
6:   if FRACTIONALBOUND( $i, curVal, curWt$ )  $\leq best$  then return ▷ prune
7:   end if
8:   if  $curWt + w_i \leq W$  then EXPLORE( $i+1, curVal + v_i, curWt + w_i$ )
9:   end if
10:  EXPLORE( $i+1, curVal, curWt$ )
11: end function
```

We branch on the order in which cities are appended to a partial tour and prune with a *lower bound* on the cost to complete it – for instance, the cost so far plus, for the current city and every unvisited city, the cheapest edge that could still leave it. If this admissible bound already exceeds the best complete tour found, the partial tour is abandoned.

Algorithm 6 Branch-and-Bound TSP

```
1:  $best \leftarrow +\infty$ ; start the tour at city 0
2: function EXPLORE( $current, count, cost$ )
3:   if  $cost \geq best$  then return
4:   end if
5:   if  $count = n$  then
6:      $best \leftarrow \min(best, cost + d(current, 0))$ ; return
7:   end if
8:   if LOWERBOUND( $current, cost$ )  $\geq best$  then return ▷ prune
9:   end if
10:  for each unvisited city  $j$  do
11:    mark  $j$  visited; EXPLORE( $j, count + 1, cost + d(current, j)$ ); unmark  $j$ 
12:  end for
13: end function
```

The optimum must match an exact baseline such as Held–Karp DP (in $O(2^n n^2)$) or brute force; Problem 9 asks you to verify this on small n .

7 A Polynomial Island: 2-SAT

Not every restriction of a hard problem stays hard. **2-SAT** – SAT where every clause has exactly two literals – is solvable in *linear* time, even though 3-SAT is NP-complete. The trick is the **implication graph**: a clause $(a \vee b)$ is equivalent to the two implications $(\neg a \Rightarrow b)$ and $(\neg b \Rightarrow a)$. Build a directed graph with two nodes per variable (x and $\neg x$) and these implication edges.

Theorem 7.1 (Aspvall–Plass–Tarjan). *A 2-CNF formula is satisfiable iff no variable x has both x and $\neg x$ in the same strongly connected component (SCC) of the implication graph. If satisfiable, a satisfying assignment can be read off in linear time from the reverse topological order of the SCCs.*

Proof idea. Implications are transitive, so within an SCC all literals must take the same truth value. If x and $\neg x$ are forced equal, the formula is contradictory. Otherwise, processing SCCs in reverse topological order and setting each not-yet-assigned literal to **true** (and its negation **false**) never violates an implication, because an implication $u \Rightarrow w$ cannot point from a later (true) SCC to an earlier (false) one. $\square$

Computing SCCs via Kosaraju’s or Tarjan’s algorithm gives the whole procedure in $O(n + m)$. Problem 10 asks you to implement this and return an assignment or **None**.

8 State-Space Search: Iterative Deepening

Many PSPACE-flavored puzzles (sliding tiles, word ladders) reduce to finding a path in an enormous, implicitly-defined state graph. Breadth-first search finds a shortest path but needs memory proportional to the frontier – often infeasible. **Iterative-deepening DFS (IDDFS)** combines BFS’s optimality (shortest path in number of moves) with DFS’s small memory footprint: run a depth-limited DFS with limit $1, 2, 3, \dots$ until the goal is found.

Algorithm 7 Iterative-Deepening DFS

```

1: for  $\ell = 1, 2, 3, \dots$  do
2:   run depth-limited DFS from the start with depth limit  $\ell$ 
3:   if the goal is reached then return the path
4:   end if
5: end for

```

Although shallow levels are re-explored, the cost is dominated by the deepest level (a geometric series), so the total work is within a constant factor of a single DFS to the solution depth, while space stays $O(\ell)$. Problem 11 asks you to solve a small sliding puzzle with IDDFS and return the solution path.

9 Set Cover by FPT/Bitmask Search

Definition 9.1 (Set Cover). *Given a universe U and a family of subsets $S_1, \dots, S_m \subseteq U$, find the fewest subsets whose union is U .*

Set Cover is NP-hard, but parameterized by the solution size k it admits a bounded-search-tree algorithm: pick any still-uncovered element e ; some chosen set must contain e , so branch over the (often few) sets containing e , decrementing the budget. Representing covered elements as a *bitmask* makes union and coverage tests $O(1)$ machine operations. Problem 12 asks you to implement this search parameterized by solution size and verify it against brute force.

10 Summary and Looking Ahead

This week extended our map of the computational landscape in two directions.

- **PSPACE** captures polynomial-*space* computation and contains NP. Its complete problem, **QSAT/QBF**, has alternating $\exists/\forall$ quantifiers – the logic of two-player games. **Game trees** are evaluated by **minimax**, sped up (without changing the answer) by **alpha-beta pruning**.

- **Fixed-parameter tractability** confines exponential cost to a parameter k : **bounded search trees** give Vertex Cover in $O(2^k n)$ (Theorem 4.4), and **kernelization** shrinks an instance to size depending only on k .
- **Special structure** makes hard problems easy: Independent Set is linear-time on **trees** via DP, and **2-SAT** is linear-time via the implication-graph/SCC method even though 3-SAT is NP-complete.
- **Branch-and-bound** solves Knapsack and TSP *exactly* by pruning with relaxation bounds, and **iterative-deepening** searches huge state spaces with little memory.

The throughline: *NP-hardness is a worst-case verdict, not a death sentence*. Real instances often have a small parameter, special structure, or bounds tight enough to prune the search dramatically.

Next week (**Approximation Algorithms and Local Search**) we relax a different requirement: instead of guaranteeing an *exact* optimum quickly, we will design polynomial-time algorithms that guarantee a solution provably *close* to optimal (approximation ratios), and study local-search heuristics that iteratively improve a solution toward a local optimum.